KI in TYPO3

Probleme, Fragen und Anforderungen

Erfahrungen mit nr-llm und darauf aufbauenden Anwendungen

Sebastian Mendel · Netresearch
9. November 2026



Was euch heute erwartet

01 Möglichkeiten: fünf Anwendungen erleben

02 Fragen zum KI-Einsatz: Qualität, Aufwand, Daten und Verantwortung

03 Antworten und Grenzen: am Beispiel nr-llm

04 Agentenwege: intern und extern vergleichen

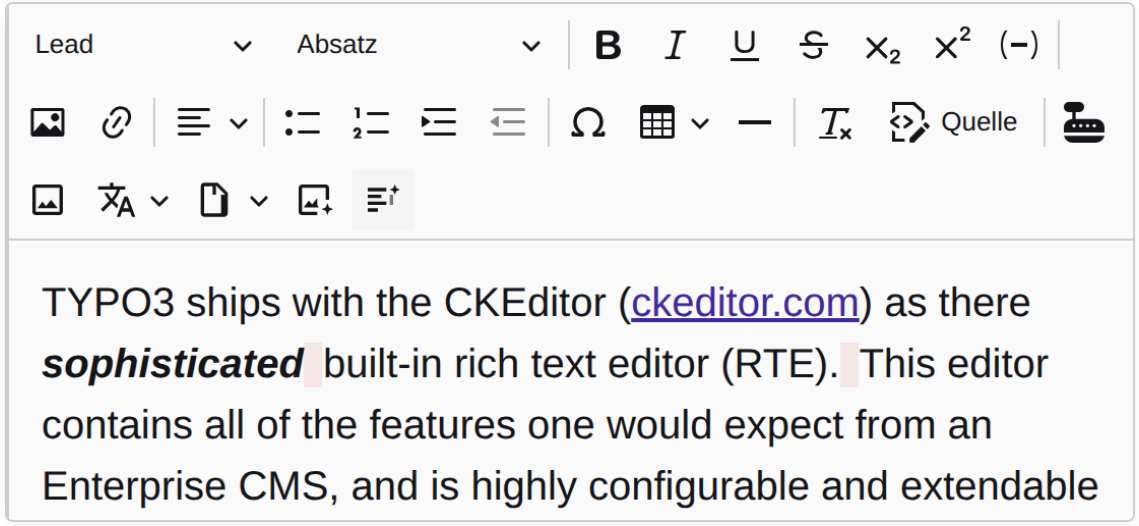
05 Einordnung: andere Grundlagen und Austausch

01

Was ist mit KI in TYPO3 möglich?

KI direkt beim Bearbeiten nutzen

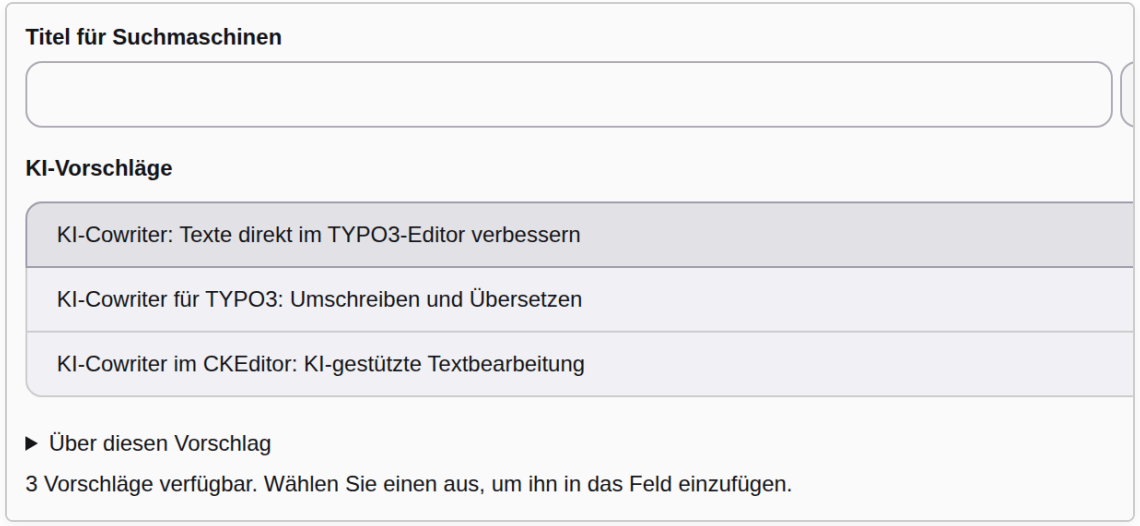
Im Editor



Die KI-Werkzeuge lassen sich direkt im Texteditor aufrufen.

DEMO Editor öffnen ↗

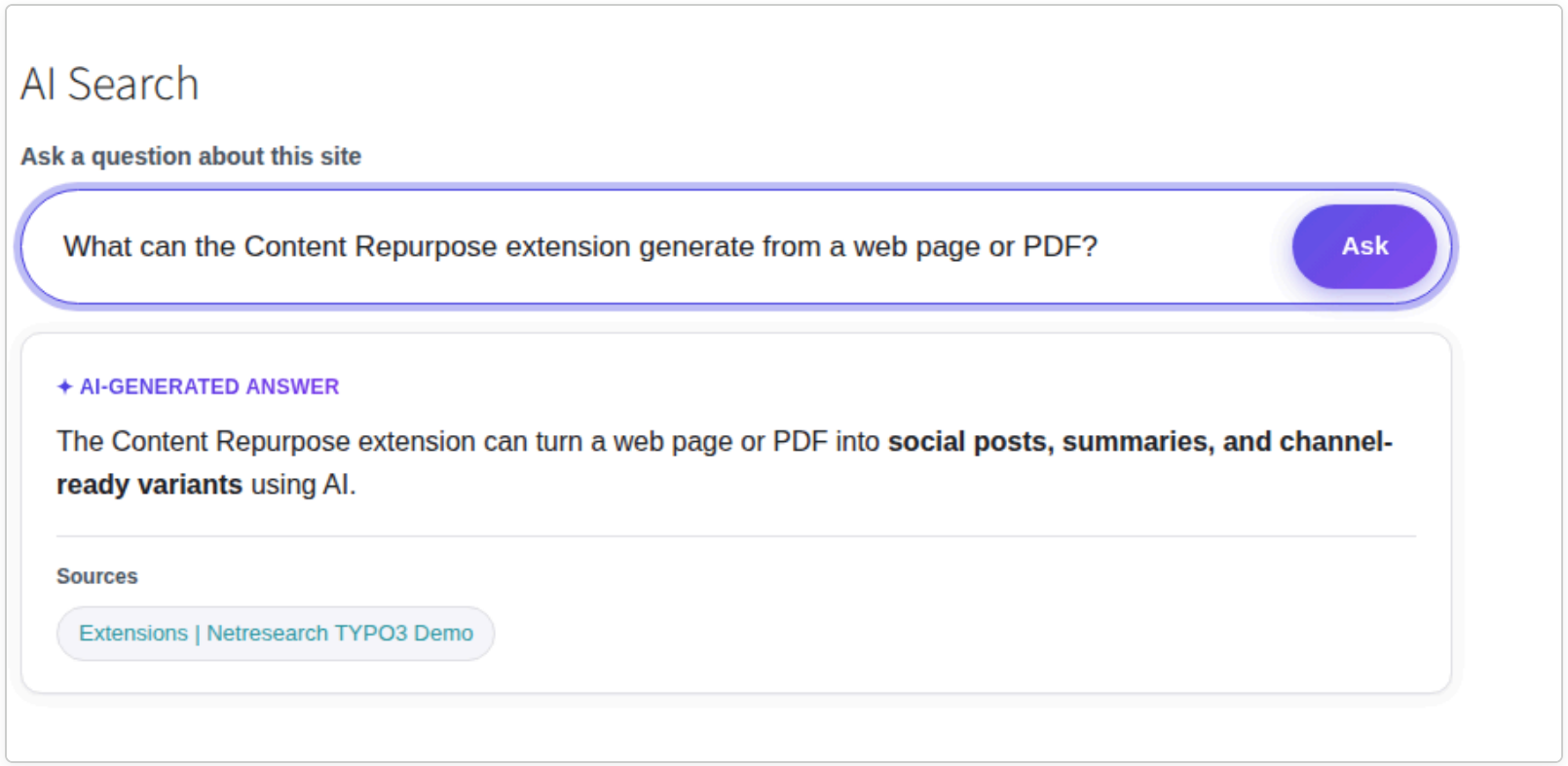
An Seitenfeldern



Für den SEO-Titel stehen mehrere Vorschläge zur Auswahl.

DEMO Seitenfelder öffnen ↗

Antworten aus Website-Inhalten



01 Antwort mit Quelle nachvollziehen

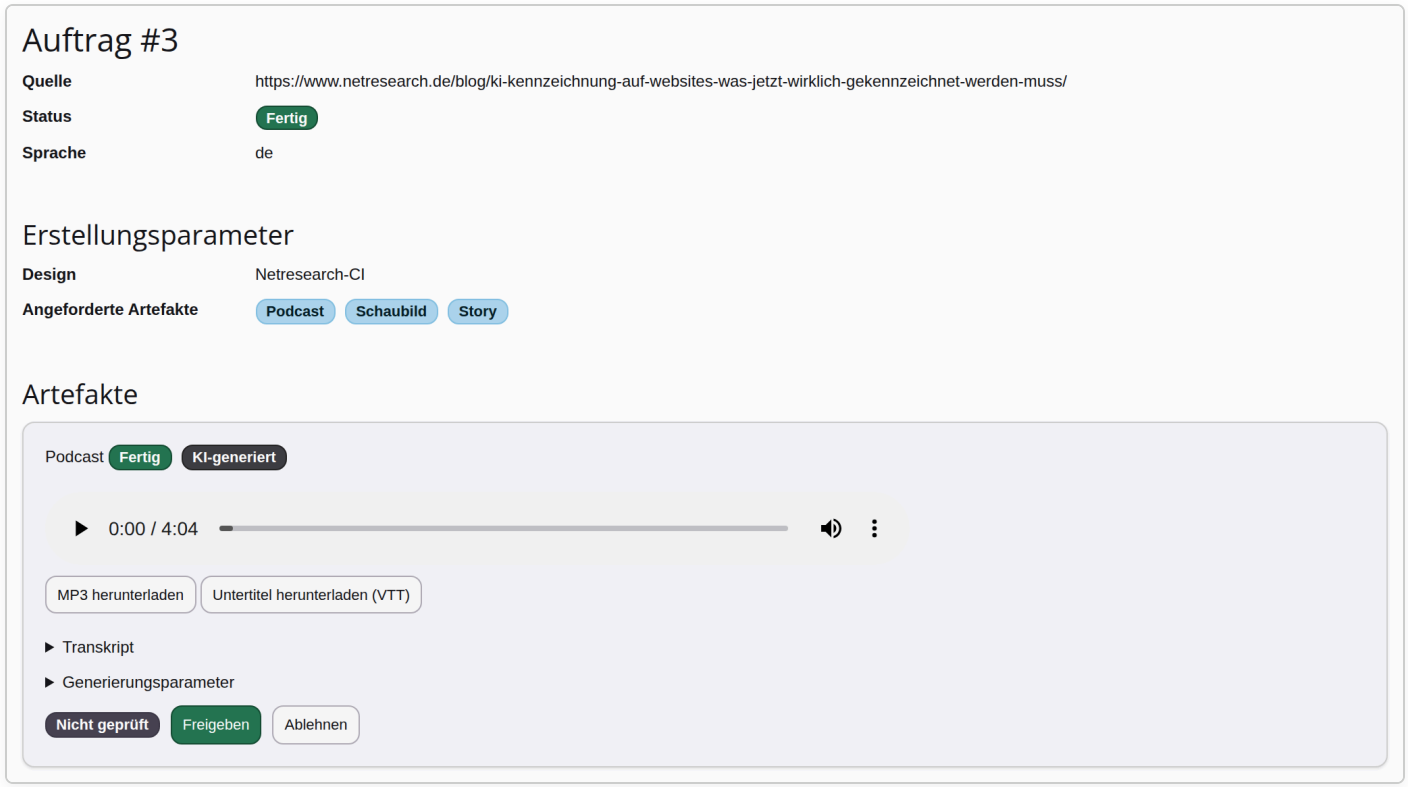
02 Quelltreffer öffnen

03 Zeigen, wenn die Quellen keine Antwort liefern

DEMO AI Search · Vorschau ↗

Die Antwort verlinkt die verwendete Quelle zum Nachlesen.

Ein Inhalt, mehrere Formate



Podcast und Schaubild nutzen dieselbe verlinkte Ausgangsquelle.

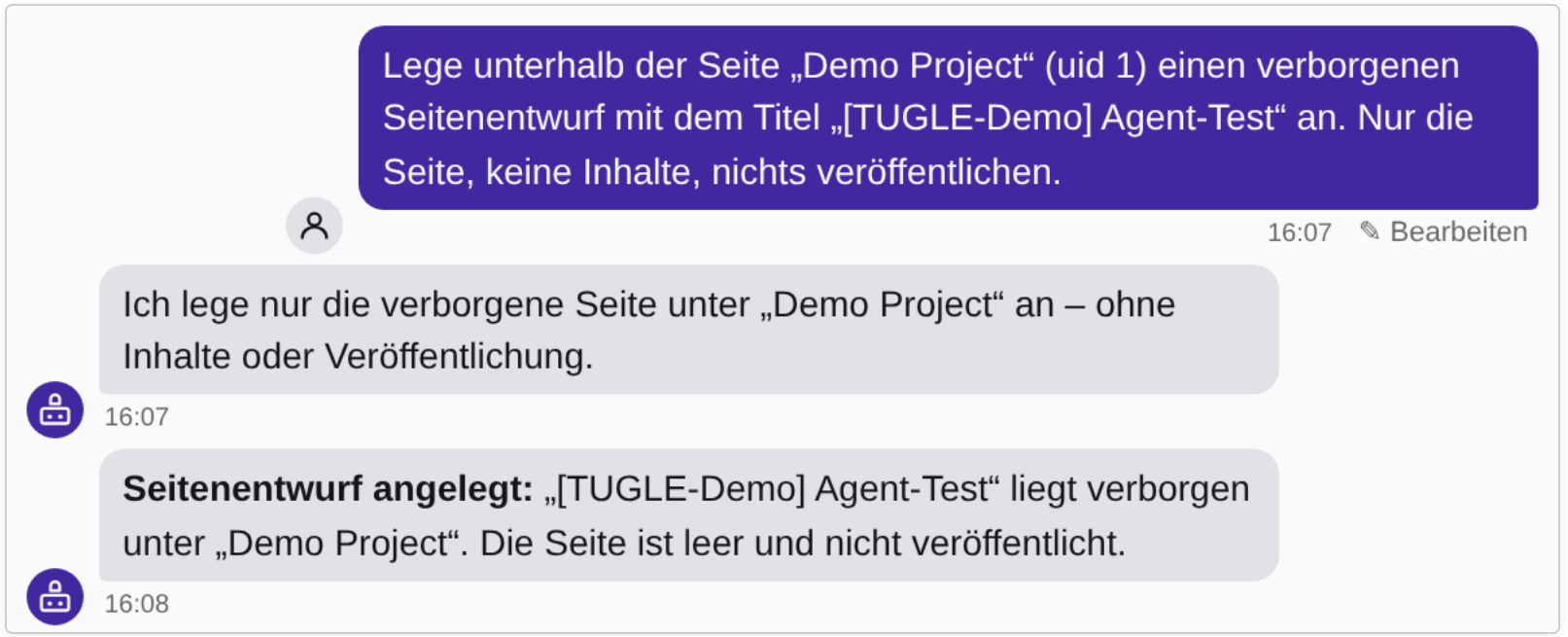
01 Ausgangsquelle bleibt nachvollziehbar

02 Schaubild und Podcast als Ergebnisse

03 Jedes Artefakt einzeln prüfen

DEMO **Repurpose öffnen ↗**

Mit TYPO3 sprechen und Aufgaben erledigen lassen



Der Chat zeigt Auftrag und Rückmeldung zum Seitenentwurf.

01 Fragen zu Inhalten in TYPO3 stellen

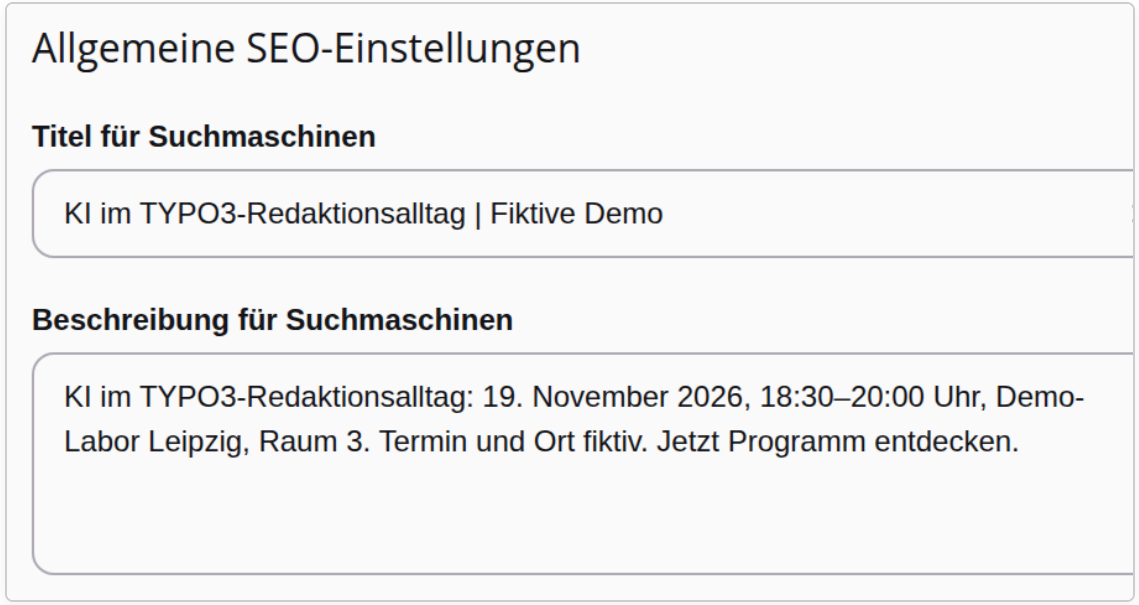
02 Änderungen in natürlicher Sprache beauftragen

03 Zum Beispiel einen Seitenentwurf anlegen lassen

DEMO KI-Chat öffnen ↗

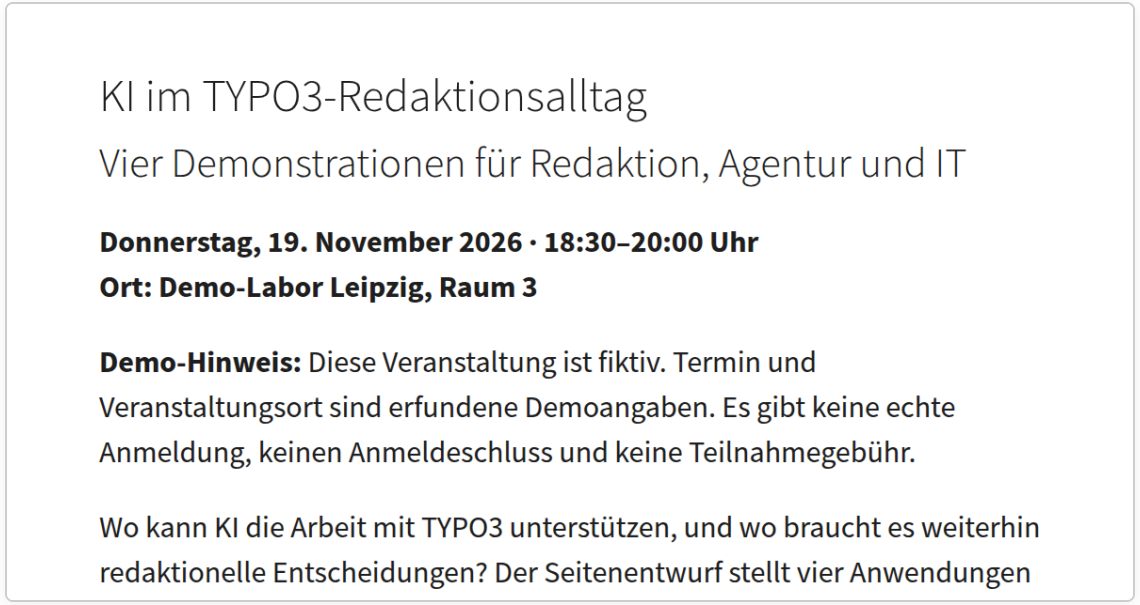
Vom Briefing zum Seitenentwurf

Im Backend



Die erzeugten Seitenfelder bleiben im Backend bearbeitbar.

In der Vorschau



Die Vorschau macht den Seitenentwurf redaktionell prüfbar.

DEMO Entwurf in Vorschau ↗

Weitere Möglichkeiten in unserer Demo

Fehler analysieren	Logs im Chat untersuchen und mögliche Ursachen eingrenzen.
Im System recherchieren	Fragen zu Inhalten, Datensätzen und Konfiguration stellen.
Externe Systeme einbinden	Über MCP Informationen und Werkzeuge nutzen – etwa DeepWiki.
Bilder und Alttexte	Bilder erzeugen und Alttexte auf Deutsch und Englisch ergänzen.
Inhalte übersetzen	Auch ganze TYPO3-Datensätze in eine andere Sprache übertragen.
Formulare bedienen	Eine Wetterabfrage in natürlicher Sprache formulieren – mit Browser AI.

02

Fragen zum KI-Einsatz

Was müssten wir vor dem Einsatz im eigenen TYPO3 klären?

Aus den gerade gezeigten Beispielen

Aus euren bisherigen Projekten

Vier Felder für unsere Fragen

Qualität

Welche Fehler lassen sich automatisch erkennen? Was müssen Menschen prüfen?

Aufwand und Betrieb

Was müssen wir integrieren und betreiben? Was kostet das?

Daten und Vorgaben

Welche Inhalte dürfen wohin? Welche rechtlichen und betrieblichen Vorgaben gelten?

Kontrolle und Verantwortung

Wer darf ändern, wer gibt frei und wer verantwortet das Ergebnis?

03

Antworten und Grenzen

Die KI-Anbindung in jeder Extension neu bauen?

Cowriter	AI Search	Repurpose	Landingpage	Backend-Agent

Integrierte Anwendungen nutzen dieselbe technische Grundlage.

nr-llm

Provider und Zugänge

Konfigurationen und Aufgaben

Nutzung und Kontrolle

Modellanbieter und Modelle

Die Fachfunktion und ihr Ablauf bleiben in der jeweiligen Extension.

PROJEKT **Modularer KI-Stack im Überblick ↗**

GITHUB **nr-llm: Projekt und Dokumentation ↗**

Modell wechseln, ohne die Anwendung neu zu bauen?

Anwendung

SEO-Text erzeugen

Ruft eine benannte Konfiguration auf.

→

Konfiguration

Vorgaben festlegen

Vorgaben, Parameter und Modellwahl.

→

Modell

Fähigkeiten wählen

Passend zur Aufgabe.

→

Provider

Dienst anbinden

Endpunkt und Zugang.

Die Anwendung behält ihre Schnittstelle; die Modellwahl wird zentral geändert.

Beim Wechsel benötigte Fähigkeiten, Datenweg und Ergebnisqualität erneut prüfen.

DEMO **Konfigurationen öffnen** ↗

Wie verwenden Aufgaben dieselben Vorgaben?

Beispielaufgabe

Absatz kürzen

Bei jedem Aufruf kommt ein anderer Text dazu.

Zentrale Anweisung in der Konfiguration

Sprache beibehalten.
Keine neuen Fakten ergänzen.

Gemeinsame Textbausteine ergänzen z. B. Tonalität oder Zielgruppe.

Die Aufgabe verbindet Eingabe und Konfiguration; gemeinsame Anweisungen werden wiederverwendet.

Eine Prompt-Anweisung ist noch kein Nachweis, dass das Ergebnis sie einhält.

DEMO **Konfigurationen öffnen** ↗

GITHUB **Tasks: Eingabe und Konfiguration** ↗

Woher kennt die KI unsere Inhalte?

In TYPO3

Benötigte Quelle

Zum Beispiel der ausgewählte Absatz im Editor.

→

In der Anwendung

Kontext zusammenstellen

Absatz + Auftrag + passende Anweisungen.

→

Beim Modell

Mit diesem Inhalt arbeiten

Für die aktuelle Anfrage.

Das Modell erhält den zusammengestellten Kontext, nicht automatisch das gesamte TYPO3.

Eine Suche liefert gefundene Inhalte; ein Agent kann über erlaubte Werkzeuge weitere Informationen laden.

Welche Bausteine ergänzen den Kontext?

Prompt-Snippets	Gemeinsame Vorgaben für Tonalität, Zielgruppe oder Format.
Skills	Wiederverwendbare Arbeitsanweisungen passend zur Aufgabe einbinden.
Ausgewählte Inhalte	Gezielt Absätze oder Datensätze aus der Anwendung mitgeben.
Wissenssuche	Quellenauszüge aus öffentlichen Website-Inhalten nachladen.
MCP-Werkzeuge	Externe Systeme über freigeschaltete Werkzeuge abfragen.

Einbindung, Freigaben und Rechte bestimmen, was tatsächlich beim Modell ankommt.

Im Projekt festlegen: Welche Anweisungen, Quellen und Werkzeuge braucht die Aufgabe?

Was lässt sich automatisch prüfen?

Schema prüfen

Erlaubte Statuswerte und maximal 160 Zeichen prüfen.

Im Beispiel ausgeführt

Eine erfundene Frist ist formal gültig.

Quellen abgleichen

DecisionService: Ist der Anmeldeschluss im Briefing belegt?

Zusätzliche Bewertung

Auftrag, Entwurf und Quellen mitgeben.

Form und Quellentreue getrennt prüfen; das Ergebnis fachlich freigeben.

Hier ist nur das Schema ausgeführt. Die Anwendung muss die Quellenbewertung aufrufen; sie kann irren.

BEISPIEL **Ausgeführtes Schema-Beispiel ↗**

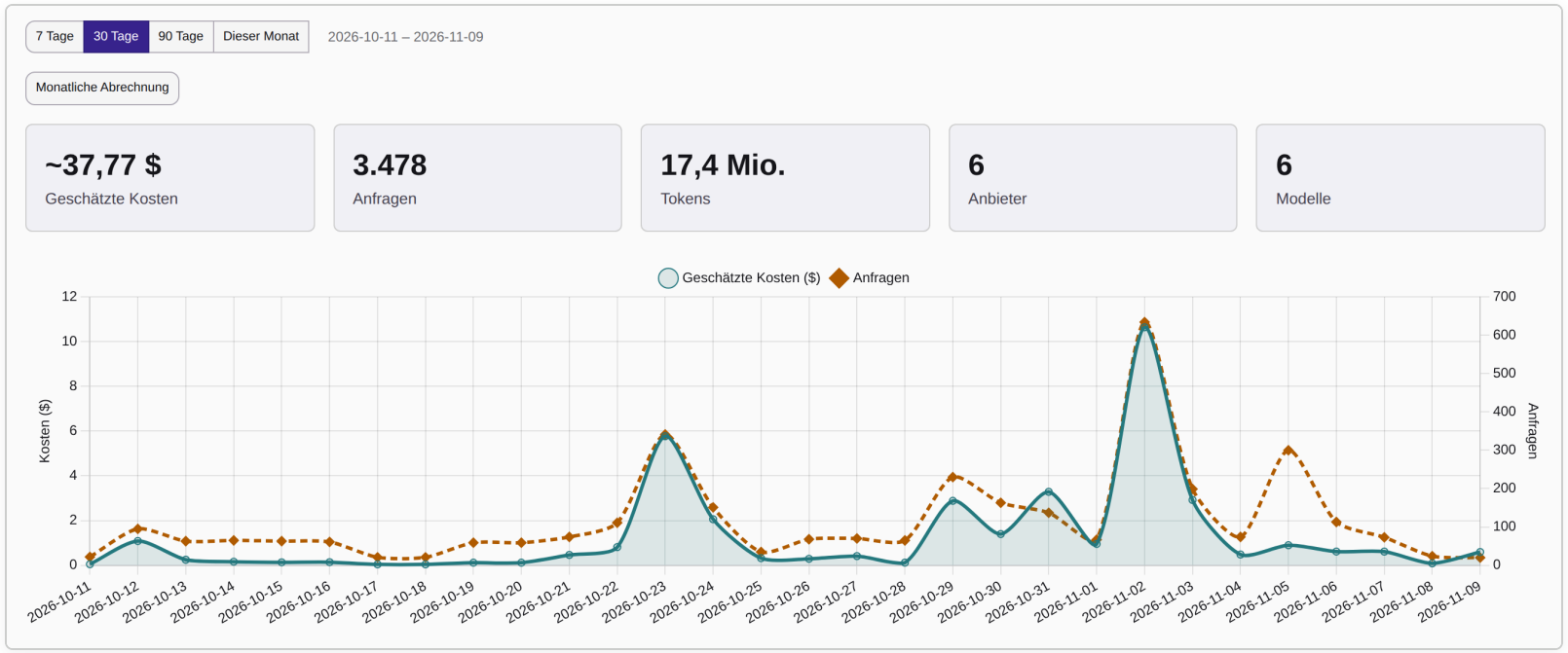
Welche Prüfungen ergänzen sich?

Struktur prüfen	Struktur, Pflichtfelder und erlaubte Werte prüfen.
Begriffe prüfen	Konfigurationsgebundene Sperr- und Pflichtwörter prüfen.
Quellen bewerten	DecisionService: Entwürfe mit Quellen und Prüffragen bewerten.
Varianten vergleichen	Varianten an festen Beispielaufgaben vergleichen.
Fachlich freigeben	Menschen prüfen in der Anwendung das fertige Ergebnis.

Welche Prüfungen laufen, hängt von der Anwendung ab. Modellurteile und Stichproben können irren.

Im Projekt festlegen: Was muss vor Verwendung oder Veröffentlichung ein Mensch prüfen?

Wie behalten wir Verbrauch und Kosten im Blick?



Die Übersicht zeigt Anfragen, Tokenverbrauch und geschätzte Kosten.

01 Aufrufe und Tokens aufschlüsseln

02 Kosten anhand hinterlegter Preise schätzen

03 Konfigurierte Budgets vor weiteren Aufrufen prüfen

DEMO **Kostenübersicht öffnen ↗**

Welche Dienste unterstützen den Betrieb?

Anbindung teilen	Provider und Konfigurationen in mehreren Anwendungen nutzen.
Fehlerpause	Circuit Breaker: Nach wiederholten Anbieterfehlern vorübergehend abbrechen.
Ausweichziel	Fallback: Konfigurierte Ausweichziele bei geeigneten Fehlern versuchen.
Budgets prüfen	Konfigurierte Grenzen vor weiteren Anfragen prüfen.
Verbrauch auswerten	Aufrufe und Tokens erfassen; Kosten anhand hinterlegter Preise schätzen.

Parallele Aufrufe können Budgetgrenzen überschreiten. Kosten bleiben Schätzwerte.

Im Pilot messen: Integration, laufende Betreuung, Ergebnisqualität und Nacharbeit.

Darf der Agent Quellcode an diesen Anbieter senden?

AUFTRAG „Erkläre mir diese TYPO3-Klasse.“

Werkzeug

Quellcode lesen

`read_source`
greift auf Projektcode zu.

→

Datenklasse

Quellcode

`sourceCode`
ist am Werkzeug hinterlegt.

→

Provider-Zone

Globaler Anbieter

`externalGlobal`
ist dafür nicht zulässig.

STOPP Bei **enforce** wird der Werkzeugaufruf blockiert.
Das gilt auch für Administratoren.

Beispiel: fest zugeordneter Provider, korrekt deklarierte Zone.
Freitext erhält nicht automatisch eine Datenklasse.

Was schützt Daten zusätzlich?

Schlüssel maskieren	Erkennbare Geheimnismuster in Prompt- und Antworttext ersetzen.
Zugänge schützen	nr-vault: API-Zugänge über geschützte Referenzen beziehen.
Web-Werkzeug absichern	Ziele und Weiterleitungen prüfen; standardmäßig Zustimmung verlangen.
Werkzeugzugriff begrenzen	Gruppen, einzelne Werkzeuge und Nutzerrechte einschränken.
Run-Protokolle begrenzen	Speicherumfang und Fristen wählen; Löschlauf einrichten.

Maskierung erkennt keine beliebigen sensiblen Inhalte. Toolargumente und Embeddings sind ausgenommen.

Im Projekt klären: erlaubte Nutzung, Anbietervereinbarungen und Aufbewahrung.

Wer darf Änderungen ausführen?



Der Agent legt die geplante Seitenänderung zur Freigabe vor.

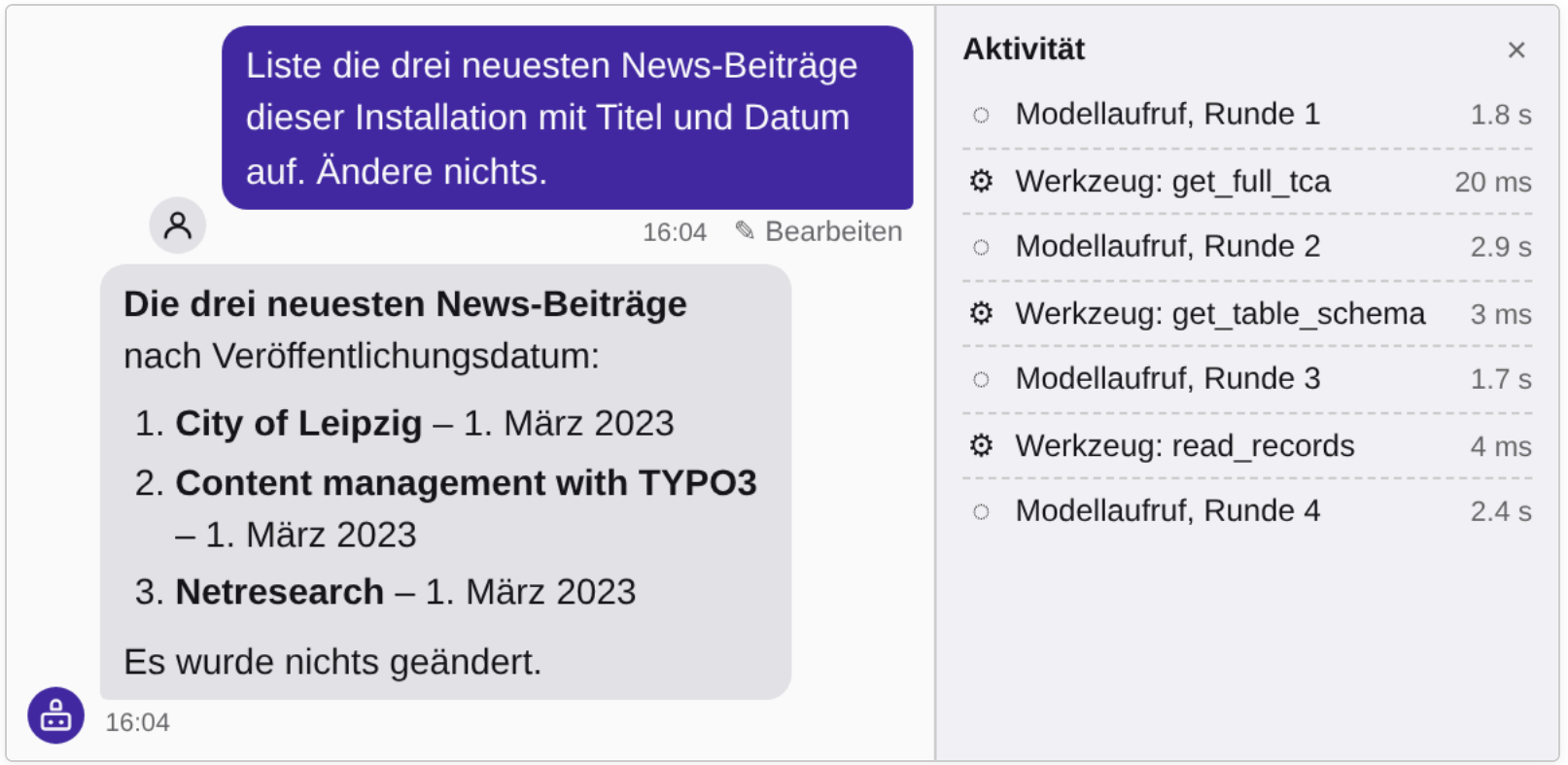
01 Lesende und schreibende Werkzeuge unterscheiden

02 Backend-Agent: konkrete Änderung vorab freigeben

03 Schreibende Standardwerkzeuge prüfen TYPO3-Rechte

DEMO Werkzeuge öffnen ↗

Was hat der Agent tatsächlich getan?



01 Werkzeug, Argumente und Ergebnis ansehen

02 Freigaben und ausgeführte Schritte nachvollziehen

03 Aus Protokollen gezielt Fehler untersuchen

DEMO KI-Chat öffnen ↗

Werkzeugaufrufe machen den Weg zur Antwort nachvollziehbar.

Welche Sicherungen ergänzen die Freigabe?

Rechte prüfen	Eingebaute Werkzeuge prüfen die Rechte des handelnden Nutzers.
Vieraugenfreigabe	Optional einen anderen Backend-Nutzer freigeben lassen.
Vorschau binden	Aufruf und angezeigte Werkzeugvorschau vor Ausführung erneut prüfen.
Freigabe protokollieren	Schreibfreigaben nur mit gespeichertem Freigabeereignis ausführen.
Eigene Regeln	Fachliche Bedingungen unmittelbar vor Werkzeugaufrufen prüfen.

Die Zustandsbindung braucht eine erfolgreiche Vorschau; eigene Regeln ergänzt die Anwendung.

Im Projekt festlegen: Wer erlaubt Aktionen, prüft Ergebnisse und verantwortet die Veröffentlichung?

04

Agentenwege

Wie kann ein Agent mit TYPO3 arbeiten?

Im TYPO3-Backend

Auftrag im Backend
↓ Agent in TYPO3
↓ TYPO3-Werkzeuge

Für Arbeit direkt in TYPO3.

Im eigenen Assistenten

Auftrag im externen Assistenten
↓ Agent und TYPO3-MCP-Server
↓ TYPO3-Werkzeuge

Für Abläufe im vorhandenen Assistenten.

MCP standardisiert den Werkzeugzugriff.

Welche Rechte und Freigaben greifen, hängt von Agent und MCP-Server ab.

DOKU **MCP-Architektur nachlesen ↗**

GITHUB **Beispiel: TYPO3 MCP Server ↗**

A2A: Ein Agent beauftragt einen anderen

01

Auftrag übergeben

Der Benutzer-Agent beauftragt einen TYPO3-Agenten.

→

02

Aufgabe bearbeiten

Der TYPO3-Agent arbeitet im eigenen Zuständigkeitsbereich.

→

03

Ergebnis zurückgeben

Der beauftragende Agent erhält eine Rückmeldung.

Ausblick: Identität, Rechte und Rückmeldung müssen zwischen den Agenten geklärt sein.

DOKU **A2A und MCP: Einordnung lesen ↗**

05

Einordnung & Austausch

Welche weiteren Grundlagen gibt es für TYPO3?

KI-Schicht für Extensions

Gemeinsame Anbindung für eigene KI-Funktionen.

GITHUB Netresearch · nr-llm ↗

GITHUB b13 · AiM (Alpha) ↗

DOKU NITSAN / T3Planet · AI Foundation ↗

WEB LOUIS INTERNET · LIA Middleware AI ↗

Für Extensions und Eigenentwicklungen · kommerziell

MCP-Zugang zu TYPO3

TYPO3-Werkzeuge für externe Assistenten.

GITHUB hauptsache.net · TYPO3 MCP Server ↗

GITHUB Marek Skopal · TYPO3 MCP Server ↗

GITHUB AutoDudes · AI Suite MCP (Zusatz) ↗

KI-Anbindung und Agentenzugriff sind unterschiedliche Aufgaben; beide Bausteine können sich ergänzen.

Vier Antworten für den eigenen Einstieg

Qualität

Automatische Prüfungen und menschliche Freigabe kombinieren. Kriterien je Aufgabe festlegen.

Aufwand und Betrieb

Integration, Betrieb und Nacharbeit im Pilot messen. Tokenkosten sind nur ein Teil.

Daten und Vorgaben

Erlaubte Datenwege, Anbieter und Aufbewahrung festlegen. Technische Sperren gezielt nutzen.

Kontrolle und Verantwortung

Rechte und Freigaben festlegen. Verantwortliche für Ergebnis und Veröffentlichung benennen.

Eine Aufgabe wählen, mit Beispielen prüfen und erst bei messbarem Nutzen erweitern.

Welche Fragen sind noch offen?

01 Welche eurer gesammelten Fragen ist noch offen?

02 Wo bleiben Zweifel oder konkrete Hürden?

03 Welche Aufgabe würdet ihr als Erstes ausprobieren?

Links zu Demos und Anwendungen

Präsentation und Demo

VORTRAG Aktuelle neutrale Präsentation ↗

GITHUB Bildnachweise und Aufnahmedaten ↗

DEMO Öffentliche TYPO3-Demo ↗

DEMO TYPO3-Backend · Anmeldung nötig ↗

DEMO AI Search · Backend-Vorschau ↗

DEMO Landingpage-Entwurf · Backend-Vorschau ↗

ARTIKEL Ausgangsartikel des Repurpose-Beispiels ↗

Anwendungen und Beschreibungen

GITHUB Cowriter ↗

DEMO AI Search · Demo-Übersicht ↗

GITHUB Repurpose ↗

GITHUB Backend-Agent ↗

GITHUB Landingpage ↗

Quellen zu KI-Grundlagen und MCP

Technische Grundlage

PROJEKT Modularer KI-Stack · Module und Voraussetzungen ↗

GITHUB nr-llm · Projekt und Dokumentation ↗

GITHUB nr-llm 0.43.19 · geprüft am 6.11.2026 ↗

DOKU MCP · Architektur von Client und Server ↗

DOKU A2A und MCP · offizielle Einordnung ↗

DOKU Symfony AI Platform · PHP-Unterbau ↗

KI-Schichten und MCP-Server

GITHUB b13 AiM · TYPO3-KI-Schicht ↗

DOKU NITSAN / T3Planet · AI Foundation ↗

WEB LOUIS INTERNET · LIA Middleware AI ↗

DOKU Madj2k AI Core + TYPO3 AI Assistant ↗

GITHUB hauptsache.net · TYPO3 MCP Server ↗

GITHUB Marek Skopal · TYPO3 MCP Server ↗

GITHUB AutoDudes · AI Suite MCP (Zusatz) ↗

Rückfragen: die passende Erklärung öffnen

Entwicklung

Formale Vorgaben 37
Quellenprüfung 54
Begriffslisten 58
Skills einbinden 39
Eigene Extension 42
Kontext und Wissen 43
Modellwechsel 45
Such- und RAG-Grundlagen 53
AI Search und Repurpose 57

Integration

MCP-Werkzeuge 40
Fehler und Kontextgrenzen 44
Circuit Breaker und Fallback 59
Datenklassen 55
Grenzen der Datenprüfung 56
Maskierung und Vault 60
Web-Werkzeug 61
Protokolle und Löschfristen 47
KI-Schichten vergleichen 51

Entscheidung

Qualität evaluieren 38
Agentenablauf 41
Werkzeugrechte 46
Gebundene Freigaben 62
Eigene Regeln 63
Nutzen und Aufwand 48
Kosten und Budgets 49
Produktivstart 50
MCP-Server vergleichen 52

Formale Vorgaben werden maschinell geprüft

01

Schema festlegen

Pflichtfelder, Datentypen, erlaubte Werte und Längen definieren.

→

02

Antwort erzeugen

Die Anwendung übergibt das Schema an `completeStructured()`.

→

03

Lokal prüfen

nr-llm prüft die Antwort gegen das unterstützte Schema.

→

04

Nur bei Fehlern

Ein Reparaturversuch; danach gültiges Ergebnis oder Fehler.

Prüfbeispiel: erlaubte Statuswerte und höchstens 160 Zeichen für die Beschreibung.

Ein gültiges Format belegt noch keine korrekte Aussage.

BEISPIEL **Ausgeführtes Prüfbeispiel ↗**

GITHUB **Funktionsweise und Quellbeleg ↗**

Qualität wird an Beispielaufgaben gemessen

01

Testfälle festlegen

Typische Aufgaben, erwartete Aussagen und Referenzantworten sammeln.

→

02

Antworten bewerten

Feste Regeln prüfen; optional bewertet der DecisionGrader über den DecisionService.

→

03

Läufe vergleichen

Bestehensquote und Bewertungen machen Verschlechterungen sichtbar.

Beispiel: Enthält die Antwort die belegten Öffnungszeiten und das erwartete Format?

Evaluationen laufen gesondert. Eine Prüfung im Arbeitsablauf muss die Anwendung zusätzlich einbinden.

GITHUB **Funktionsweise und Quellbeleg ↗**

Skills werden als Prompt-Kontext eingebunden

01

Quelle laden

Skill im Backend pflegen oder aus GitHub synchronisieren; Version festhalten.

→

02

Inhalt freigeben

Version und Herkunft prüfen; gewünschte Skills freigeben.

→

03

Aufgabe zuordnen

Skills zuordnen oder freigegebene Skills bei Bedarf laden.

→

04

Text einfügen

Freigegeben + vertrauenswürdig: Systemnachricht. Ungeprüfte Always-Skills: User-Kontext.

Auf Abruf zugeordnete Skills benötigen Freigabe; Prozessskills einen ausdrücklichen Aufruf.

Skill-Text liefert Kontext. Zugehörige Skripte und Dateien werden nicht ausgeführt oder nachgeladen.

GITHUB **Funktionsweise und Quellbeleg ↗**

MCP verbindet den Agenten mit Werkzeugen

01

Modell schlägt vor

Wählt ein angebotenes Werkzeug und formuliert seine Argumente.

→

02

nr-llm prüft

Nur Backend-Administratoren; zusätzliche Rechte und Freigaben begrenzen den Aufruf.

→

03

MCP-Client ruft auf

Sendet den Werkzeugaufruf per HTTP an den externen Server.

→

04

Ergebnis fließt zurück

Textresultat oder Fehler wird Kontext für den nächsten Modellschritt.

Vorher: Server konfigurieren, Werkzeugkatalog importieren und einzelne Werkzeuge aktivieren.

In diesem Ablauf ist TYPO3 der MCP-Client. Der externe Server führt das Werkzeug aus.

GITHUB **Funktionsweise und Quellbeleg ↗**

Ein Agent arbeitet in einer begrenzten Schleife

01

Auftrag

Ziel, Konfiguration, Skills und erlaubte Werkzeuge zusammenstellen.

→

02

Modell

Antwortet direkt oder schlägt einen Werkzeugaufruf vor.

→

03

Freigabe

Bei freigabepflichtigen Aufrufen pausieren und Entscheidung abwarten.

→

04

Werkzeug

Zulässigen Aufruf ausführen und Ergebnis zum Verlauf ergänzen.

→

05

Nächste Runde

Modell erhält das Ergebnis und bestimmt den nächsten Schritt.

↶ Das Werkzeugergebnis geht zurück an die Modellentscheidung – bis zur Antwort oder einer Grenze.

Das Modell schlägt Schritte vor; nr-llm steuert Ausführung, Pausen und Grenzen.

GITHUB **Funktionsweise und Quellbeleg ↗**

Wie bindet eine eigene Extension nr-llm ein?

01

Service injizieren

Passendes nr-llm-Interface über TYPO3 Dependency Injection verwenden.

→

02

Aufruf vorbereiten

Eingaben, Berechtigungen und die gewünschte Konfiguration bestimmen.

→

03

Ergebnis anfordern

Direkter Feature-Service oder wiederkehrender Task mit Prompt-Vorlage.

→

04

Ergebnis verarbeiten

Fehler behandeln, Ausgabe prüfen und fachliche Freigabe umsetzen.

Beispiel: completeForConfiguration($prompt, $configuration) liefert eine CompletionResponse.

nr-llm übernimmt die Modellanbindung. Die Extension gestaltet den fachlichen Ablauf.

GITHUB **Schnittstellen und Quellbeleg ↗**

Woher kommen Kontext und Wissen?

Mitgegeben

Prompt, aktuelle Inhalte und Skilltexte kommen aus der Anwendung.

Nachgeschlagen

Suchwerkzeuge liefern Auszüge mit Quellen aus vorhandenen TYPO3-Suchen.

Separat gepflegt

Indexaktualisierung und dauerhafte Vektorsuche bleiben eigene Integrationsaufgaben.

Das Modell sieht bereitgestellte oder abgerufene Quellen – keinen automatischen Wissensbestand.

GITHUB **Schnittstellen und Quellbeleg ↗**

Was passiert bei Fehlern oder zu viel Kontext?

Verbindung gestört

Begrenzt erneut versuchen; je nach Fehler eine konfigurierte Ersatzverbindung nutzen.

Zugriff falsch

Authentifizierung und Konfiguration korrigieren; Fehler an die Anwendung melden.

Kontext zu groß

Alte Chat-Runden können entfallen. Einen Einzelprompt muss die Anwendung passend aufteilen.

Die Anwendung braucht einen Fehlerpfad; Wiederholung und Fallback lösen nicht jedes Problem.

GITHUB **Schnittstellen und Quellbeleg ↗**

Ein Modellwechsel braucht einen Eignungstest

Anbindung wechseln

Die Anwendung nutzt eine benannte Konfiguration. Dahinter lassen sich Modell und Anbieter ändern.

Eignung prüfen

Benötigte Fähigkeiten, Kontextgröße und Parameter prüfen; typische Aufgaben erneut testen.

Rückfall erproben

Ersatzkonfigurationen bei Ausfällen testen. Datenziel, Qualität und Kosten jedes Ersatzes prüfen.

Eine gemeinsame Schnittstelle macht Modelle nicht fachlich austauschbar.

GITHUB **Modellwahl und Fähigkeiten ↗**

GITHUB **Rückfall im Quellcode ↗**

Werkzeugzugriff braucht mehrere Freigaben

01

Werkzeug aktivieren

Gruppe und einzelnes Werkzeug zentral freischalten.

→

02

Aufgabe begrenzen

Gruppen je Konfiguration und Lauf begrenzen; einzelne Werkzeuge ausschließen.

→

03

Nutzerrechte prüfen

Je Werkzeug: Adminpflicht oder konkrete TYPO3-Rechte prüfen.

→

04

Änderung bestätigen

Bei den eingebauten Schreibwerkzeugen die geplante Änderung vorab freigeben.

Zusätzlich gelten Datenklasse und Provider-Zone: read_source wird bei externalGlobal + enforce gesperrt.

Eine Zustimmung ersetzt weder Nutzerrechte noch Werkzeug- und Datengrenzen.

GITHUB **Werkzeugrechte im Quellcode** ↗

GITHUB **Schreibrechte und Freigaben** ↗

Betrieb braucht Messwerte und Löschroutinen

Nutzung auswerten

Anfragen, Tokens und geschätzte Kosten nach Anbieter, Modell und Nutzer ansehen.

Läufe nachvollziehen

Metadaten zu Status, Schritten, Modellaufrufen und Entscheidungen in der Run-Timeline prüfen.

Aufbewahrung betreiben

Speicherumfang und Fristen wählen. Den Löschlauf regelmäßig ausführen lassen.

Kosten sind Schätzungen. Eine Löschfrist wirkt erst durch einen geplanten Löschlauf.

GITHUB **Run-Timeline und Grenzen ↗**

GITHUB **Aufbewahrung und Löschlauf ↗**

Wann lohnt sich eine gemeinsame KI-Grundlage?

Wiederkehrende Arbeit

Eine konkrete Aufgabe fällt regelmäßig an. Inhalte und Arbeitsablauf liegen bereits in TYPO3.

Gemeinsame Steuerung

Mehrere Anwendungen brauchen dieselben Anbieter, Modelle, Budgets oder Freigaberegeln.

Messbarer Nutzen

Im Pilot Zeit inklusive Nacharbeit, Ergebnisqualität und laufende Kosten mit dem bisherigen Ablauf vergleichen.

Einführen, wenn der gemessene Nutzen Integration und Betrieb rechtfertigt.

PROJEKT **Module und Voraussetzungen vergleichen ↗**

Wie begrenzen wir Kosten?

Vor dem Aufruf

nr-llm prüft konfigurierte Nutzer- und Konfigurationslimits. Übergebene Plankosten fließen in die Prüfung ein.

Nach dem Aufruf

nr-llm erfasst die gemeldete Nutzung. Geschätzte Kosten stammen aus der Antwort oder den Modellpreisen.

Im Betrieb ergänzen

Preise und Anbieterrechnung abgleichen. Für eine harte Kostengrenze zusätzlich eine Reservierung oder Sperre einplanen.

Parallele Aufrufe können das Budget überschreiten. Die Vorabprüfung garantiert keine harte Kostengrenze.

GITHUB **Budgets und ihre Grenzen ↗**

Was muss vor dem produktiven Einsatz geklärt sein?

Datenfluss

Welche Daten gehen wohin? Anbieter, Region und Aufbewahrung für Modell, Ersatzpfad und Werkzeuge festlegen.

Rechte und Freigabe

Wer darf lesen, schreiben und veröffentlichen? Zuständige Person und Freigabeschritt je Ablauf benennen.

Qualität

Typische Aufgaben und kritische Fehler testen. Vorab festlegen, welche Ergebnisse Menschen prüfen müssen.

Betrieb

Verantwortung für Kosten, Protokolle und Updates zuweisen. Abschaltung und Ablauf bei Ausfall festlegen.

Produktiv starten, wenn Datenwege, Freigaben, Qualitätsmaßstab und Betriebsverantwortung feststehen.

PROJEKT **Datenflüsse und Verantwortung im Überblick ↗**

Welche gemeinsame KI-Schicht passt zur Integration?

Grundlage	Schwerpunkt	Für die Auswahl prüfen
GITHUB Netresearch nr-llm ↗	Dienste, Konfigurationen, Tasks und Werkzeuge	Passende API und Regeln für die eigene Anwendung
GITHUB b13 AiM · Alpha ↗	KI-Proxy, Routing und Regeln; Symfony-AI-Bridges	Benötigte Bridges und Änderungen der Alpha-API
DOKU T3Planet AI Foundation ↗	Service-API, Provider, Prompts, Kontext und MCP	Integration der eigenen Extension und Betriebsmodell
WEB LIA Middleware AI ↗	Zentrale KI-Anbindung für Extensions und Eigenentwicklungen	Kommerziell; API und Lizenz beim Hersteller klären
DOKU Madj2k AI Core + AI Assistant ↗	PHP-Kern und TYPO3-Integration für Assistenz, Pipelines und RAG	Passung zum eigenen Such- oder Assistenzablauf

An einer konkreten Aufgabe vergleichen: Schnittstelle, Datenweg, Rechte, Kosten und Wartung.

MCP-Server: welcher Zugang zu TYPO3?

hauptsache.net

Eigenständiger Server. Datensätze über Workspaces bearbeiten; eigene PHP-Werkzeuge ergänzen.

Marek Skopal

Eigenständiger Server. Direkte Verwaltung als Hauptmodus; Workspaces optional und TCA-Werkzeuge erweiterbar.

AutoDudes MCP

Zusatz zur AI Suite. TYPO3-Inhalte und ausgewählte KI-Funktionen über die Dienste und Rechte der Suite anbieten.

MCP gibt Assistenten Werkzeugzugriff. KI-Anbindung, Rechte und Freigaben bleiben eigene Entscheidungen.

GITHUB hauptsache.net · MCP-Server ↗

GITHUB Marek Skopal · MCP-Server ↗

Welche Grundlagen gibt es für eigene Wissenssuche?

EXT:index

Indexierungsframework für TYPO3-Inhalte: Aufbereitung und Weitergabe über Ereignisse oder Webhooks.

SEAL + seal_ai

Suchabstraktion, Embeddings und Vektorspeicher; optional lesender MCP-Suchzugang. seal_ai: Proof of Concept.

smart_search

Dienste für Vektoren, semantische Suche und RAG in eigenen Extensions. Als Alpha gekennzeichnet.

Diese Bausteine kümmern sich um Wissenszugriff. Eine KI-Schicht allein pflegt noch keinen Suchindex.

GITHUB Indexierungsframework ↗ | GITHUB SEAL / Symfony AI ↗ | GITHUB RAG-Bausteine ↗

Wie prüft nr-llm die Treue zur Quelle?

01

Anwendung liefert

Auftrag, fertigen Entwurf und Originalquellen; dazu konkrete Prüffragen.

→

02

nr-llm bewertet

Der DecisionService lässt ein Modell die Kriterien bewerten und liefert ein strukturiertes Ergebnis.

→

03

Anwendung entscheidet

Schwellenwerte, Überarbeitung und menschliche Freigabe bleiben Teil des Arbeitsablaufs.

Beispiel: „Ist der 4. November in der Quelle belegt?“ Im Schema-Beispiel wurde diese Bewertung nicht ausgeführt.

Seit 0.39 als allgemeiner Dienst verfügbar. Auch ein Prüfergebnis ist kein Wahrheitsbeweis.

GITHUB **DecisionService und Einbindung ↗**

Welche Daten dürfen zu welchem Anbieter?

Datenklasse	Global extern	EU extern	Privat gehostet	Lokal
Öffentliche Inhalte	Ja	Ja	Ja	Ja
Redaktionelle Inhalte	Ja	Ja	Ja	Ja
Quellcode	Nein	Ja	Ja	Ja
Interne Konfiguration	Nein	Ja	Ja	Ja
Betriebsdaten / Logs	Nein	Nein	Ja	Ja
Zugangsdatennahe Daten	Nein	Nein	Nein	Ja

Festgelegte Policy: read_source und search_code gehören zur Klasse sourceCode. Bei externalGlobal + enforce sind sie auch für Admins gesperrt.

Der Modellname entscheidet nicht über die Zone. Lokaler Open-Source-Code wird dabei genauso eingestuft wie vertraulicher Projektcode.

GITHUB **Zonen und Zuordnung ↗** GITHUB **Werkzeugklassen ↗**

Wo liegen die Grenzen der Datenprüfung?

Deklaration

Freitext wird nicht automatisch klassifiziert. Falsche Angaben zu Datenklasse oder Provider-Zone werden nicht inhaltlich erkannt.

Werkzeuge

Eigene Deklaration vor Gruppenklasse; sonst strengste Klasse. MCP übernimmt die Serverklasse und erzwingt sie auch bei observe.

Tatsächliches Ziel

Fallbacks zählen mit. Bei dynamischer Modellwahl können manche Aufrufpfade für Prüfung und Versand getrennt wählen.

Pfade mit übergebener ModelResolution, darunter der DecisionService, binden Prüfung und Versand an dieselbe Auflösung.

Ohne gültige Datenklasse kein MCP-Import. Die Policy erkennt weder sensible Inhalte noch den tatsächlichen Datenstandort.

GITHUB Dynamische Modellwahl ↗

GITHUB MCP-Klassifizierung ↗

Wie nutzen Anwendungen die Quellenprüfung?

AI Search

Eine Anti-Halluzinationsprüfung ist dokumentiert. Welchen nr-llm-Dienst sie intern aufruft, ist hier nicht verifiziert.

Repurpose: Prüfpunkte

PR 205: Briefing gegen das Original prüfen. Danach jeden fertigen Formattext erneut gegen das Original abgleichen.

Repurpose: Ablauf

Nach Kürzungen prüfen, vor Audio- und Medienproduktion. Automatische Bewertung getrennt von der redaktionellen Freigabe führen.

Im veröffentlichten Repurpose 0.14.0 fehlt der semantische Quellenabgleich. PR 205 implementiert ihn, ist aber noch nicht zusammengeführt oder veröffentlicht.

„Nicht geprüft“ und „Prüfung fehlgeschlagen“ dürfen nicht wie ein bestandenes Ergebnis erscheinen.

GITHUB Repurpose 0.14.0 · geprüfter Stand ↗

GITHUB PR 205 · Implementierung ↗

Was können Begriffslisten prüfen?

Geltung festlegen

Listen den gewünschten Konfigurationen zuordnen. Ohne Zuordnung prüft diese Liste den Aufruf nicht.

Begriffe abgleichen

Sperrbegriffe dürfen nicht vorkommen. Pflichtbegriffe müssen in geeigneten Textantworten enthalten sein.

Ausgabe stoppen

Ein Verstoß blockiert die geprüfte Ausgabe. Im Agentenlauf wird ein betroffener Schreibaufruf nicht angeboten oder ausgeführt.

Beispiel: „Rabatt*“ erfasst auch „Rabattaktion“. Pflichtwörter gelten nicht für Kurztexte unter 30 Wörtern oder JSON.

Grenze: Keine Erkennung von Umschreibungen. Im Agentenlauf werden Schreibargumente geprüft; Chattext und Pflichtwörter sind ausgenommen.

GITHUB Begriffslisten und Ausnahmen ↗

Was passiert, wenn ein Anbieter ausfällt?

Fehlerpause

Der Circuit Breaker zählt geeignete Anbieterfehler. Nach der eingestellten Schwelle brechen weitere Aufrufe vorübergehend schnell ab.

Ersatzweg

Ein konfigurierter Fallback versucht ein anderes Ziel bei Verbindungsfehlern, Überlastung, Serverfehlern oder offenem Circuit Breaker.

Erholung prüfen

Nach der Pause wird ein Probeaufruf zugelassen. Erfolg schließt die Sperre; ein erneuter Fehler öffnet sie wieder.

Voreinstellung: nach fünf passenden Fehlern eine Pause von 30 Sekunden. Ausweichziele müssen eingerichtet sein.

Grenze: Keine Verfügbarkeitsgarantie. Anmelde- und Konfigurationsfehler lösen keinen automatischen Ersatzweg aus.

GITHUB **Circuit Breaker und Fallback ↗**

GITHUB **Voreinstellungen ↗**

Wie werden Zugangsdaten geschützt?

Im Vault ablegen

Eingerichtete API-Zugänge verweisen auf ein Geheimnis in nr-vault. In der Konfiguration steht dessen Kennung.

Beim Aufruf einsetzen

Der sichere HTTP-Client setzt den Zugang für den Dienstaufruf ein. Der Schlüssel gehört nicht in die Modellanweisung.

Text maskieren

nr-llm ersetzt erkennbare Geheimnismuster in Prompt, Antwort und Thinking. Diese Maskierung ist verpflichtend.

Beispiel: Den API-Schlüssel als Vault-Referenz hinterlegen, statt ihn in den Systemprompt zu kopieren.

Grenze: Nur bekannte Muster werden erkannt. Toolargumente und Embeddings sind ausgenommen; personenbezogene Inhalte werden nicht allgemein erkannt.

GITHUB Zugänge über nr-vault ↗

GITHUB Maskierung und Grenzen ↗

Wie liest der Agent eine externe Webseite?

Bewusst aktivieren

Das Web-Werkzeug ist zunächst ausgeschaltet. Nach Aktivierung verlangt ein Aufruf standardmäßig menschliche Zustimmung.

Adresse prüfen

Startadresse und jede Weiterleitung durchlaufen die Adressprüfung. Private Zieladressen werden auch mit Hostfreigabe abgewiesen.

Text begrenzen

Das Werkzeug liefert begrenzten Text und kennzeichnet ihn als nicht vertrauenswürdiges Material. Es führt kein Webseiten-JavaScript aus.

Beispiel: Eine öffentliche Produktseite lesen; deren Weiterleitung auf eine private Netzwerkadresse wird abgewiesen.

Grenze: Auch URL-Parameter können Daten übertragen. Öffentlich erreichbare interne Dienste brauchen zusätzliche Sperrregeln; der Schutz gilt für dieses Werkzeug.

GITHUB **Web-Werkzeug und Adressprüfung ↗**

Woran ist eine Schreibfreigabe gebunden?

Andere Person

Bei aktivierter Vieraugenregel darf der auslösende Backend-Nutzer nicht selbst freigeben. Das gilt auch für Administratoren.

Geprüfter Stand

Die Freigabe muss zu den anstehenden Aufrufen passen. Eine inzwischen geänderte Vorschau verlangt eine neue Entscheidung.

Gespeicherter Nachweis

Vor einem Schreibschritt muss das Freigabeereignis gespeichert sein. Misslingt das, wird der freigegebene Schritt nicht ausgeführt.

Beispiel: Ein angezeigter Ausgangswert ändert sich während der Freigabe. Der Agent wartet mit einer neuen Vorschau erneut.

Grenze: Vieraugen gilt für Backend-Nutzer. Die Zustandsbindung braucht eine erfolgreiche Werkzeugvorschau; Freigabe ersetzt keine fachliche Prüfung.

GITHUB Vieraugenregel ↗ GITHUB Vorschaubindung ↗ GITHUB Aufruf und Audit ↗

Wie ergänzt das Projekt eigene Werkzeugregeln?

01

Regel entwickeln

Die Anwendung implementiert eine eigene fachliche Bedingung über die Schnittstelle für Werkzeugregeln.

→

02

Vor Ausführung prüfen

Die Regel bewertet endgültige Argumente, handelnden Nutzer und Ziel. Auch nach menschlicher Freigabe wird erneut geprüft.

→

03

Entscheidung anwenden

Eine ablehnende Regel verhindert den Aufruf. Fehler in einer Regel führen ebenfalls zur Ablehnung.

Projektbeispiel: Diese Automatisierung darf nur Datensätze im freigegebenen Seitenbereich ändern.

Grenze: Ohne eigene Regel entsteht keine zusätzliche Fachprüfung. Benötigte, aber fehlende Zielangaben muss die Regel ausdrücklich ablehnen.

GITHUB Eigene Werkzeugregeln ↗